Lab 10 ESP32 Bluetooth Communication

ให้นิสิตส่งข้อมูลจาก ESP32 มายังคอมพิวเตอร์ด้วย Bluetooth Classic หรือ Bluetooth Low Energy

ต่อยอดจาก Lab 8 และ Lab 9 โดยคงวงจรและฟังก์ชันเดิม ได้แก่ POT, LED, Switch, OLED, Preferences และ SPIFFS จากนั้นเพิ่มช่องทาง Bluetooth เพื่อส่งสถานะและรับคำสั่งจากคอมพิวเตอร์ นิสิตเลือกทำได้ 1 แนวทางหลัก คือ Bluetooth Classic หรือ Bluetooth Low Energy

ภาพรวมงานที่ต้องทำ

ให้นิสิตเลือกทำอย่างน้อย 1 ส่วน โดยยังคงการทำงานพื้นฐานของ Lab 8 ให้ไม่เสียไป หากทำได้ทั้ง 2 ส่วนให้เปรียบเทียบผลการทดลองในรายงาน

ส่วน	ชื่องาน	ผลลัพธ์ที่ต้องเห็น
10A	Bluetooth Classic Serial Communication	คอมพิวเตอร์รับสถานะจาก ESP32 เป็นข้อความ และส่งคำสั่งกลับเพื่อควบคุมบอร์ดได้
10B	Bluetooth Low Energy Web Dashboard	คอมพิวเตอร์เปิดไฟล์ HTML แล้วเชื่อมต่อ BLE เพื่ออ่าน Status, รับ Notify, ส่ง Command และอ่าน log.csv ได้

10A Bluetooth Classic Serial Communication

ส่วนนี้ให้นิสิตใช้ Bluetooth Classic เป็นช่องทางสื่อสารแบบ Serial ระหว่าง ESP32 กับคอมพิวเตอร์ เหมาะสำหรับเริ่มต้นเพราะเห็นข้อมูลเป็นข้อความและ Debug ได้ง่าย

การเชื่อมต่อ

- อัปโหลดโปรแกรม ESP32 ที่เปิด Bluetooth Classic และตั้งชื่ออุปกรณ์เป็น ESP32-LAB10A-<StudentID>
- Pair คอมพิวเตอร์กับ ESP32 ผ่าน Bluetooth Settings
- เปิด Serial Bluetooth Terminal หรือโปรแกรม Serial Port ที่เห็น COM Port ของ ESP32
- ตั้ง Baud rate ใดก็ได้ตามโปรแกรม Terminal เพราะข้อมูลจริงวิ่งผ่าน Bluetooth SPP ไม่ใช่ UART จริง แต่ควรระบุค่าให้ตรงกันในคู่มือการทดลอง
- ตรวจสอบว่า ESP32 ส่งข้อมูลสถานะออกมาทุก 1 วินาที และรับคำสั่งจากคอมพิวเตอร์ได้

ข้อมูลที่ ESP32 ต้องส่งมายังคอมพิวเตอร์

ให้ส่งเป็น JSON หนึ่งบรรทัดต่อหนึ่งตัวอย่าง เพื่อให้อ่านง่ายและนำไปประมวลผลต่อได้ เช่น

```
{"lab":"10A","student":"65123456","adc":2048,"voltage":1.65,"led":true,"pwm":128,"logging":false,"logCount":25,"uptime":120}
```

Field	ความหมาย
lab	ระบุส่วนของแลป เช่น 10A
student	รหัสนิสิตหรือรหัสกลุ่ม
adc	ค่าจาก POT ช่วง 0-4095
voltage	แรงดันโดยประมาณจาก ADC
led	สถานะ LED เชิงตรรกะ เปิดหรือปิด
pwm	ค่า PWM 0-255
logging	สถานะการบันทึกข้อมูลลง SPIFFS
logCount	จำนวนรายการข้อมูลที่บันทึก
uptime	เวลาทำงานของ ESP32 หน่วยวินาที

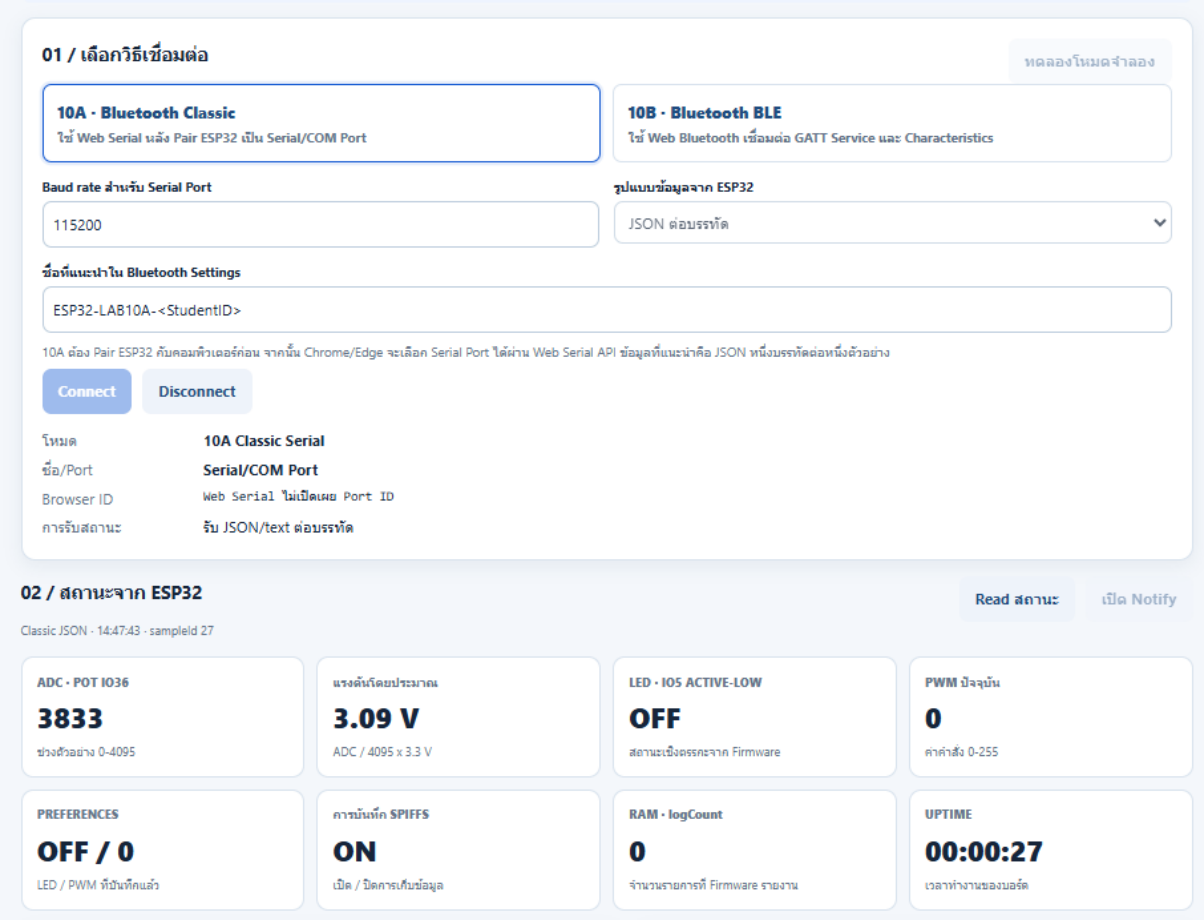
คำสั่งที่คอมพิวเตอร์ส่งกลับไปยัง ESP32

ให้รับคำสั่งเป็นข้อความหนึ่งบรรทัด จบด้วย Enter เพื่อ Debug ง่าย ตัวอย่างคำสั่งที่ต้องรองรับ

คำสั่ง	หน้าที่
LED 1	เปิด LED
LED 0	ปิด LED
PWM 128	ตั้งค่า PWM เป็น 128
LOG 1	เริ่มบันทึกข้อมูลลง SPIFFS
LOG 0	หยุดบันทึกข้อมูล
SAVE	บันทึกค่า LED และ PWM ปัจจุบันลง Preferences
READLOG	ส่งข้อมูล log.csv กลับมายังคอมพิวเตอร์
DELLOG	ลบ log.csv และรีเซ็ต logCount

ข้อกำหนดสำคัญ: เมื่อ Bluetooth หลุดหรือยังไม่เชื่อมต่อ ระบบ ADC, LED, PWM, Switch, OLED, Preferences และ SPIFFS ต้องยังทำงานต่อได้ตามปกติ ห้ามให้การรอ Bluetooth ทำให้ loop หลักค้าง

ตัวอย่าง



10B Bluetooth Low Energy Web Dashboard

ส่วนนี้ให้นิสิตใช้ BLE เพื่อเชื่อมต่อจากหน้าเว็บ lab10_ble_dashboard.html ที่แนบให้ ESP32 ทำหน้าที่เป็น Peripheral และ GATT Server ส่วนเว็บบนคอมพิวเตอร์เป็น Central และ GATT Client

การเปิดใช้งาน Dashboard

- เปิด Bluetooth ของคอมพิวเตอร์
- เปิดไฟล์ lab10_ble_dashboard.html ด้วย Chrome หรือ Edge บนคอมพิวเตอร์
- กรอก UUID ให้ตรงกับ Firmware หรือใช้ UUID ตัวอย่างที่กำหนดในแลป
- กด Scan & Connect แล้วเลือกอุปกรณ์ ESP32
- ตรวจสอบสถานะ Read/Notify, ข้อมูล ADC, LED, PWM, Logging และปุ่มควบคุม

BLE GATT ที่ใช้ในแลป

รายการ	UUID ตัวอย่าง	Properties	ทิศทาง
Service	8e7c0001-6c7d-4b2f-9f5a-1d2e3f4a5b60	Primary Service	รวม Characteristic ของ Lab 10
Status Characteristic	8e7c0002-6c7d-4b2f-9f5a-1d2e3f4a5b60	Read, Notify	ESP32 ส่งสถานะไปยังเว็บ
Command Characteristic	8e7c0003-6c7d-4b2f-9f5a-1d2e3f4a5b60	Write	เว็บส่งคำสั่งไปยัง ESP32
Log Characteristic	8e7c0004-6c7d-4b2f-9f5a-1d2e3f4a5b60	Notify	ESP32 ส่ง log.csv แบบแบ่งแพ็กเก็ตไปยังเว็บ

คำแนะนำเรื่อง UUID: Service UUID และ Characteristic UUID ไม่ควรใช้ค่าเดียวกันทั้งหมด เพราะแต่ละ Characteristic มีหน้าที่และ Properties ต่างกัน ควรใช้ UUID แยกกันตามตาราง เพื่อให้เว็บและ Firmware ระบุช่องทาง Read, Write และ Notify ได้ชัดเจน

รูปแบบข้อมูล Status สำหรับ BLE

Status Characteristic ส่งข้อมูลแบบ Binary 20 ไบต์ เพื่อให้กระชับและเหมาะกับ BLE จำนวนหลายไบต์ใช้ Little-endian

Offset	ชนิดข้อมูล	ความหมาย
0	uint8	protocolVersion = 1
1	uint8 flags	bit0 = LED เปิด, bit1 = Logging เปิด, bit2 = savedLED เปิด
2-3	uint16	adc ช่วง 0-4095
4	uint8	pwm ปัจจุบัน 0-255
5	uint8	savedPWM ใน Preferences 0-255
6-7	uint16	reserved = 0
8-11	uint32	logCount
12-15	uint32	uptimeSeconds
16-19	uint32	sampleId เพิ่มขึ้นเมื่อสร้างสถานะใหม่

รูปแบบการส่ง log.csv ผ่าน BLE เนื่องจาก BLE ส่งข้อมูลต่อแพ็กเก็ตได้จำกัด ให้แบ่งไฟล์เป็นเฟรมผ่าน Log Characteristic โดยใช้ Header 3 ไบต์ และ Payload ไม่เกิน 17 ไบต์

Type	รูปแบบ	ความหมาย
START	01 00 00	เริ่มส่งไฟล์
DATA	02 seqLo seqHi payload	ส่งข้อมูลไฟล์ตามลำดับ sequence
END	03 seqLo seqHi	จบการส่งไฟล์

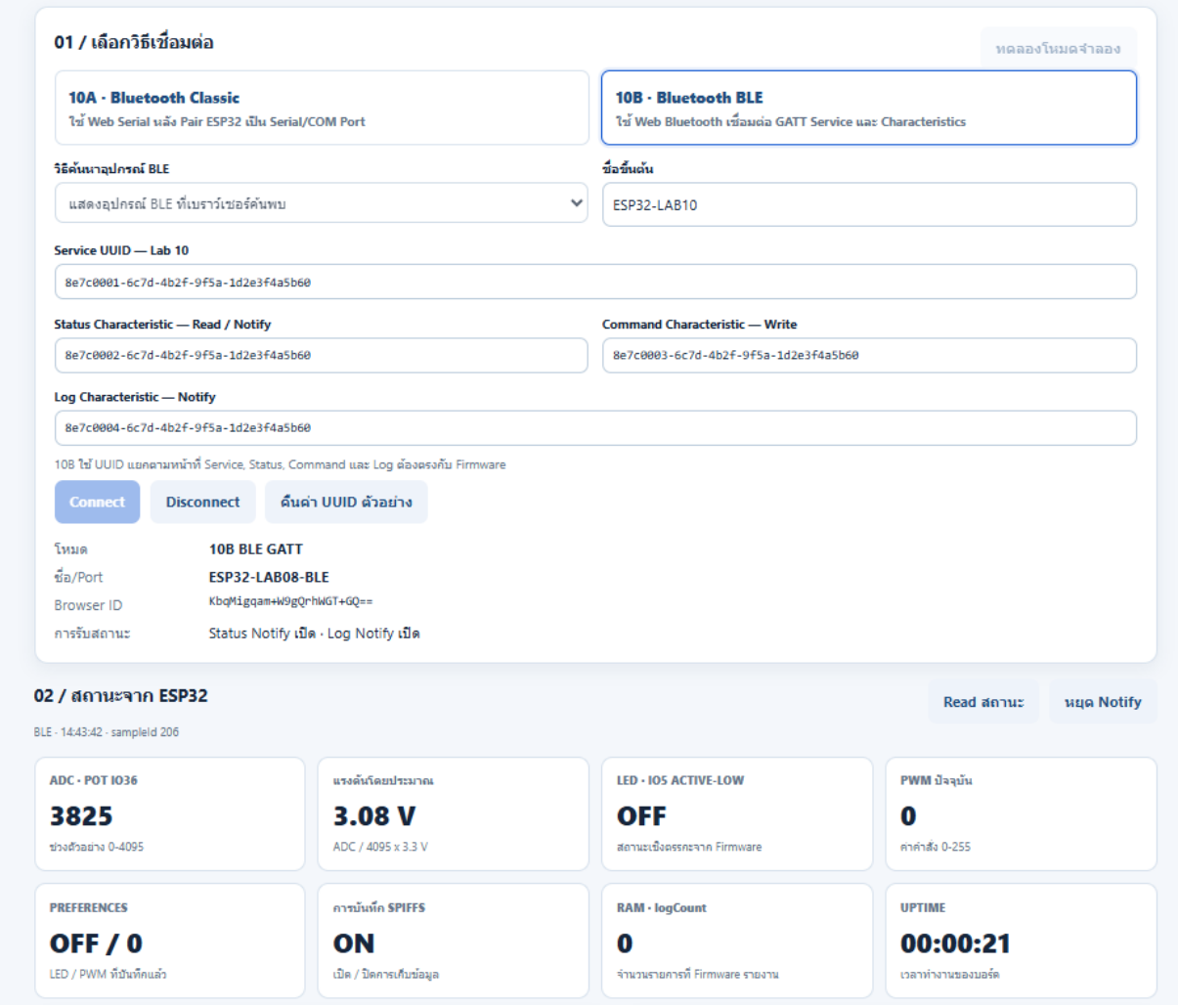
คำสั่ง BLE ที่เว็บส่งไปยัง ESP32

Command Characteristic รับข้อมูลเป็นไบต์ ไม่ใช่ข้อความ ASCII

HEX	คำสั่ง	หน้าที่
01 01	LED ON	เปิด LED เชิงตรรกะ
01 00	LED OFF	ปิด LED เชิงตรรกะ
02 xx	SET PWM	ตั้ง PWM โดย xx คือค่า 0-255
03 01	LOG ON	เริ่มบันทึกข้อมูล
03 00	LOG OFF	หยุดบันทึกข้อมูล
04	SAVE	บันทึก LED และ PWM ลง Preferences
05	READ LOG	ส่ง snapshot ของ log.csv ผ่าน Log Notify
06	DELETE LOG	ลบ log.csv และรีเซ็ต logCount

เว็บจะตรวจลำดับ sequence ก่อนรวมไฟล์ หากลำดับขาดหรือ Timeout ให้ยกเลิกการรับไฟล์และให้นิสิตลองอ่านใหม่

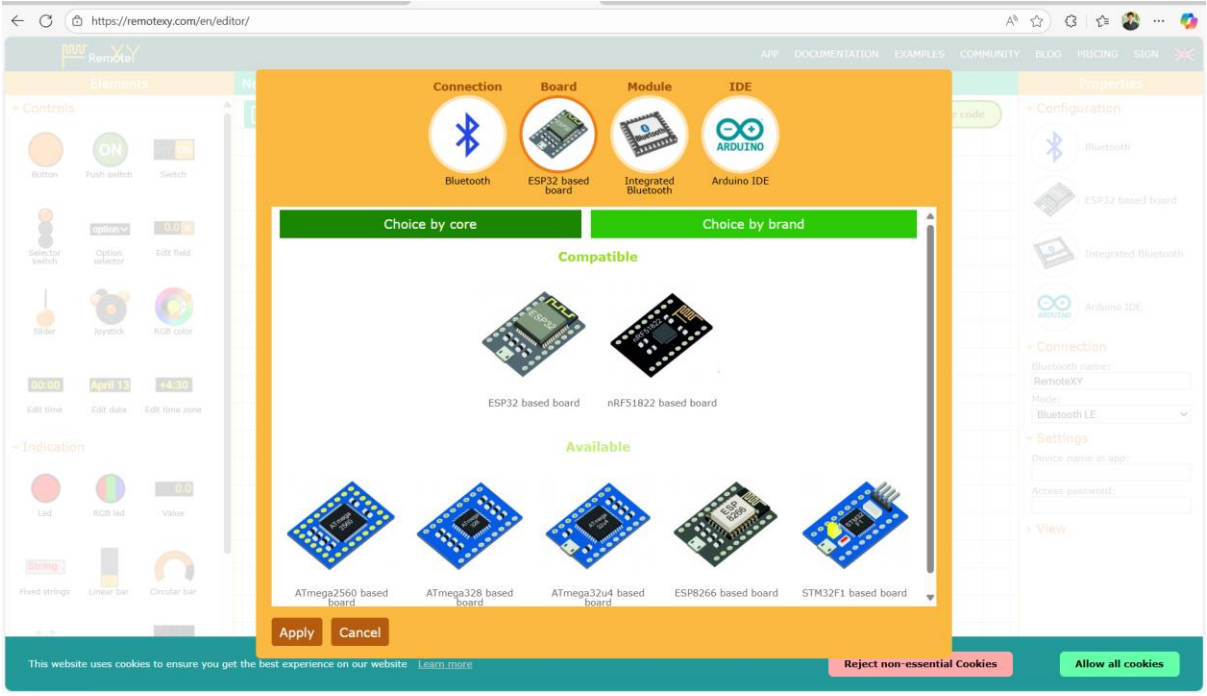
ตัวอย่าง



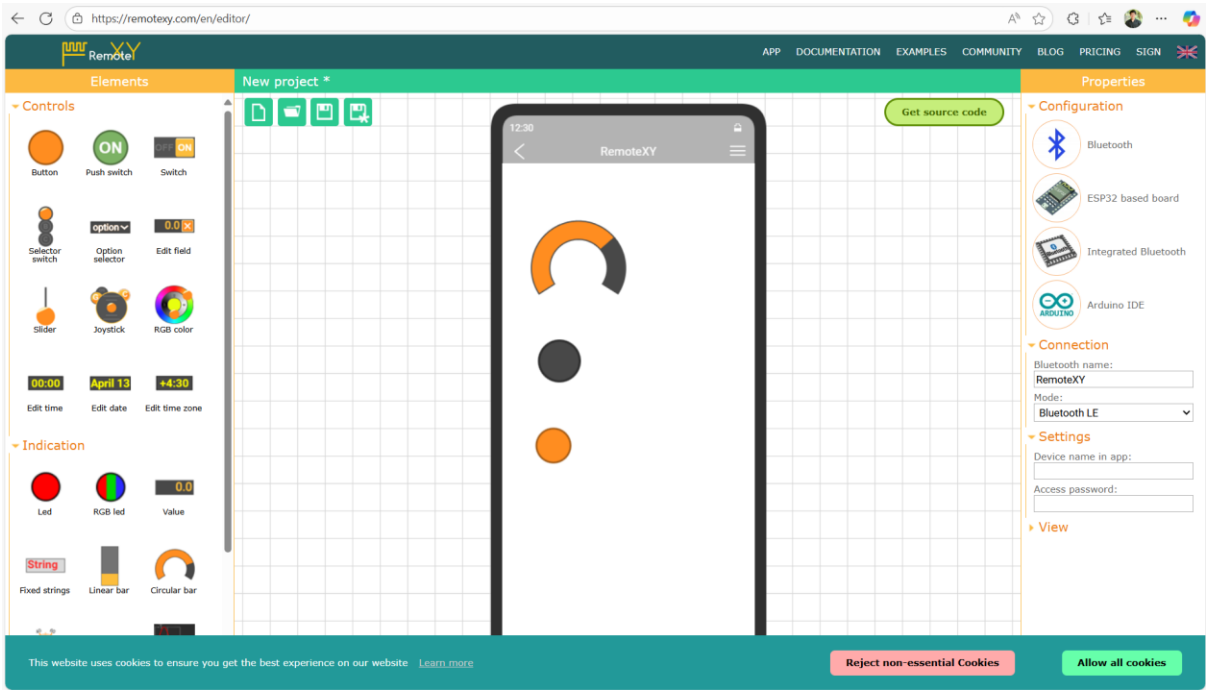
RemoteXY

Config RemoteXY เชื่อมต่อ BLE

https://remotexy.com/



ลาก Dashboard ปุ่ม Control หรือ Indicator เมื่อออกแบบเสร็จให้ Download



ESP32 Bluetooth Classic

ให้นิสิต ศึกษาการใช้งาน Bluetooth ของ ESP32 ในโหมด Bluetooth Classic

ตัวอย่างการใช้งาน Bluetooth Classic ทำให้ ESP32 สามารถรับส่งข้อมูลผ่าน Bluetooth Serial ได้

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

void setup() {
  Serial.begin(115200);
  SerialBT.begin("ESP32test"); //Bluetooth device name
  Serial.println("The device started, now you can pair it with bluetooth!");
}

void loop() {
  if (Serial.available()) {
    SerialBT.write(Serial.read());
  }
  if (SerialBT.available()) {
    Serial.write(SerialBT.read());
  }
  delay(20);
}
```

จากโค้ดด้านบน ให้ทดลองติดต่อ ESP32 ด้วย Smartphone หรือ Window

Android

App > Serial Bluetooth Terminal

Window

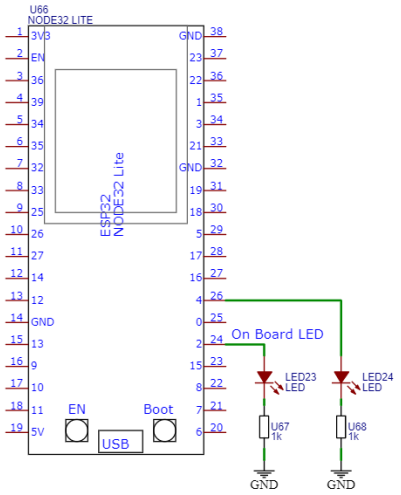
App > Bluetooth Serial Terminal

โดยให้ทดลอง

1. ส่งข้อความจาก ESP32 ว่า “Send from ESP32 Name ID” เช่น “Send from ESP32 Jirawat 139”
2. ส่งข้อความจากโปรแกรม กลับไปยัง ESP32 ว่า “Response Data Bluetooth Serial”

ESP32 Serial Bluetooth Control

ให้นิสิต นำ ESP32 ต่อวงจรร่วมกับ LED ดังรูป



ชื่อ ESP32_XXX (XXX คือ รหัสนิสิตสามตัวท้าย)

โดยกำหนดคำสั่งดังนี้

GPIO02_0 = สั่ง GPIO2 ให้ LED ON

GPIO02_1 = สั่ง GPIO2 ให้ LED OFF

GPIO04_0 = สั่ง GPIO4 ให้ LED ON

GPIO04_1 = สั่ง GPIO4 ให้ LED OFF

ควบคุม LED 2 ดวง โดยใช้ Bluetooth Serial

ถ่ายรูปผลลัพท์

1. ภาพจากโปรแกรมที่ส่งข้อความ GPIO02_1

2. ภาพของบอร์ด ESP32 ที่รับข้อมูล และ Serial Monitor ที่ได้รับข้อความดังกล่าวผ่านทาง Bluetooth

ESP32 Bluetooth Low Energy (BLE)

ให้นิสิต ศึกษาการใช้งาน Bluetooth ของ ESP32 ในโหมด Bluetooth Low Energy

ตัวอย่างการใช้งาน Bluetooth Low Energy ทำให้ ESP32 สามารถรับส่งข้อมูลผ่าน Bluetooth Low Energy ได้

```
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEServer.h>
#include <BLE2902.h>

#define SERVICE_UUID           "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_RX "33f961c8-695f-4d36-a826-fd8a9d03de4e"
#define CHARACTERISTIC_UUID_TX "29cffb82-d30d-4c7f-941b-3022cbf7c09a"

BLEServer *pServer;
BLEService *pService;
BLECharacteristic * pTxCharacteristic;
BLECharacteristic * pRxCharacteristic;
bool deviceConnected = false;
bool oldDeviceConnected = false;

class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
      deviceConnected = true;
    };

    void onDisconnect(BLEServer* pServer) {
      deviceConnected = false;
    }
};

class MyCallbacks: public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
      std::string rxValue = pCharacteristic->getValue();

      if (rxValue.length() > 0) {
        Serial.print("Received Value: ");
        for (int i = 0; i < rxValue.length(); i++)
          Serial.print(rxValue[i]);
        Serial.println();
      }
    }
};

void setup() {
  Serial.begin(115200);
  Serial.println("Starting BLE work!");

  // Create BLE Server
  BLEDevice::init("ESP32BLE");
  pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  // Create the BLE Service
  pService = pServer->createService(SERVICE_UUID);

  // Create a BLE Characteristic (TX)
  pTxCharacteristic = pService->createCharacteristic(
                        CHARACTERISTIC_UUID_TX,
                        BLECharacteristic::PROPERTY_NOTIFY|
                       BLECharacteristic::PROPERTY_READ
                      );

  pTxCharacteristic->addDescriptor(new BLE2902());

  // Create a BLE Characteristic (RX)
  pRxCharacteristic = pService->createCharacteristic(
                        CHARACTERISTIC_UUID_RX,
                        BLECharacteristic::PROPERTY_WRITE
                      );

  pRxCharacteristic->setCallbacks(new MyCallbacks());
```

```
// Start the service
  pTxCharacteristic->setValue("Hello World BLE ESP32");
  pService->start();
  BLEAdvertising *pAdvertising =
BLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  pAdvertising->setScanResponse(true);
  pAdvertising->setMinPreferred(0x06);  // functions that
help with iPhone connections issue
  pAdvertising->setMinPreferred(0x12);
  BLEDevice::startAdvertising();
  Serial.println("Characteristic defined! Now you can read
it in your phone!");
}

void loop() {

  if (deviceConnected) {
    if (Serial.available()) {
      String str = Serial.readString();
      char arrStr[100];
      str.toCharArray(arrStr, str.length());
      pTxCharacteristic->setValue(arrStr);
      pTxCharacteristic->notify();
    }
  }

  // disconnecting
  if (!deviceConnected && oldDeviceConnected) {
    pServer->startAdvertising(); // restart advertising
    Serial.println("start advertising");
    oldDeviceConnected = deviceConnected;
  }

  // connecting
  if (deviceConnected && !oldDeviceConnected) {
    oldDeviceConnected = deviceConnected;
  }

  delay(500);
}
```

จากโค้ดด้านบน ให้ทดลองติดต่อ ESP32 ด้วย Smartphone หรือ Window

Android
App > nRF Connect

ios
App > nRF Connect

Window
App > nRF Connect for Desktop

โดยให้ทดลอง

1. ส่งข้อความจาก ESP32 ว่า “Send from ESP32 BLE Name ID” เช่น “Send from ESP32 BLE Jirawat 139”

2. ส่งข้อความจากโปรแกรม กลับไปยัง ESP32 ว่า “Response Data Bluetooth BLE”

ถ่ายรูปผลลัพท์ลงในกล่องคำตอบ

1. ส่งข้อความจาก ESP32 ว่า “Send from ESP32 BLE Your_name Std ID XXX”

2. ส่งข้อความจากโปรแกรม กลับไปยัง ESP32 ว่า “Response Data Bluetooth BLE”